

ICS 26-Lab0报告

hswang24

1.回答问题

1.有过多人协作经历，两个人分别写代码发到微信中，其中一个ctrl+C、ctrl+V，效率很低

2.Git 为什么要设计“暂存—提交”两个步骤？

暂存区可以让开发者先选择本次需要提交的修改，而不是把工作区中的所有内容一次性提交。提交前还可以检查暂存区中的内容，避免把无关文件或尚未完成的代码提交进去。这样可以把不同功能的修改拆分成多个清晰、独立的提交，方便以后查看历史、定位问题和回退版本。因此，“暂存—提交”两个步骤使版本管理更加灵活、安全。

3.

```
1 | git branch
```

仅列出**本地分支**，当前分支前会标记 `*`，适合快速查看当前工作环境。

```
1 | git branch -a
```

列出**所有分支**（本地分支 + 远程跟踪分支），远程分支以 `remotes/` 开头，适合需要操作远程分支的场景。

2.完成 `main.c` 文件中的 `TODO` 部分并进行一次 `commit`

修改效果及上传记录如下

GNU nano 7.2

```
#include <stdio.h>
```

```
int main()
```

```
    // @TODO: print a sentence you want.  
    printf("Hello, FDU!\n");
```

```
hswang@localhost:~$ cd lab0-gitlab-hswang24
hswang@localhost:~/lab0-gitlab-hswang24$ nano main.c
hswang@localhost:~/lab0-gitlab-hswang24$ git diff main.c
diff --git a/main.c b/main.c
index 2122766..21308e4 100644
--- a/main.c
+++ b/main.c
@@ -3,5 +3,5 @@
 int main()
 {
     // @TODO: print a sentence you want.
-    printf("Hello, world!\n");
-}
\ No newline at end of file
+    printf("Hello, FDU!\n");
+}
hswang@localhost:~/lab0-gitlab-hswang24$ git add main.c
hswang@localhost:~/lab0-gitlab-hswang24$ ^C
hswang@localhost:~/lab0-gitlab-hswang24$ ^C
hswang@localhost:~/lab0-gitlab-hswang24$ git commit -m "修改main.c"
[feature ceeb127] 修改main.c
1 file changed, 2 insertions(+), 2 deletions(-)
hswang@localhost:~/lab0-gitlab-hswang24$ |
```

3.Commit Message规范和语义化版本的简要概括

1.语义化版本控制（Semantic Versioning，简称 SemVer）是一套为解决软件开发中“依赖地狱”问题而设计的、具有明确约定的版本号命名规范。其核心目的在于，通过版本号本身的数字变化，清晰、无歧义地向开发者传递底层代码修改的性质和影响范围，从而建立起可靠、自动化的依赖管理基础。

SemVer 旨在解决的核心问题是“依赖地狱”。在复杂的软件系统中，项目通常依赖大量第三方库（包）。若版本管理不当，则会陷入两种困境：一是“版本锁死”，即依赖关系定义得过于严格，导致任何一方需要升级时，都必须强制所有有依赖方同步更新，使得升级步履维艰；二是“版本混乱”，即依赖关系定义得过于宽松，允许安装多个未来可能不兼容的版本，导致系统行为不可预测。SemVer 通过一套严格的规则，在这两种极端之间找到了平衡点，为依赖声明提供了可靠依据。

SemVer 规范的核心是一组简单而严格的版本号递增规则。它要求版本号必须采用 X.Y.Z 的格式，分别代表主版本号、次版本号和修订号。这些数字的递增并非随意，而是与代码变更对公共API的影响直接挂钩：1) 当发布仅包含向后兼容的缺陷修复时，必须递增修订号 Z（例如从 1.0.0 升至 1.0.1）；2) 当发布向后兼容的新功能或现有功能被标记为弃用时，必须递增次版本号 Y，同时将修订号归零（例如从 1.0.0 升至 1.1.0）；3) 当发布任何不向后兼容的 API 修改时，必须递增主版本号 X，同时将次版本号和修订号归零（例如从 1.0.0 升至 2.0.0）。此外，版本 0.Y.Z 被保留用于初始开发阶段，表示 API 极不稳定；而 1.0.0 的发布则标志着公共 API 的正式形成与稳定化承诺。为了适应复杂的开发流程，SemVer 还定义了扩展标识符。主要包括“先行版本号”和“版本编译信息”。先行版本号（如 -alpha, -beta.1, -rc.1）通过连字符附加在主要版本之后，表示该版本不稳定，优先级低于对应的正式版，用于测试和预览。版本编译信息（如 +build.20231001）通过加号附加，用于记录编译元数据，在比较版本优先级时会被完全忽略。规范还对版本优先级比较规则进行了细致规定，确保自动化工具能准确判断版本的新旧。

SemVer 的价值在于为开发生态提供了清晰的沟通语言和可靠的依赖管理机制。对于一个遵循 SemVer 的库，使用者可以放心地指定依赖范围（例如 ^1.2.3，表示兼容 1.2.3 及以上、但低于 2.0.0 的所有版本），因为根据约定，所有 1.X.X 的版本都将是向后兼容的。这极大地简化了依赖更新流程，降低了维护成本。文本还通过FAQ部分解答了常见实践问题，例如如何处理意外的不兼容更新（应发布修正版本并注明问题）、如何弃用功能（先在次版本中标记，再在主版本中移除）等，增强了规范的可操作性。

2.

Commit Message 的作用

规范的 Commit Message 能够：

- **提供清晰的历史记录**，方便快速浏览代码变更目的。
- **过滤特定类型的提交**（如 `git log --grep feature` 仅查看新增功能）。
- **自动生成 Change Log**，用于版本发布时说明变更内容

Commit Message 格式

每一条 Commit Message 包含三部分：

```
1  <type>(<scope>): <subject>  // Header（必需）
2  <空行>
3  <body>                        // Body（可选）
4  <空行>
5  <footer>                      // Footer（可选）
```

规则：

- 每行不超过 72 字符
- 使用英文、第一人称现在时

价值

Angular 规范的 Commit Message 通过严格的格式约定（类型+范围+描述）和工具链支持，实现代码变更的清晰记录、高效管理和自动生成发布文档

为什么学习Git?

学习Git是现代软件开发的必备技能。首先，Git提供了强大的版本控制能力，让开发者能够安全地管理代码变更历史，避免代码丢失或冲突。其次，它支持团队高效协作，多人可以并行开发而互不干扰。更重要的是，Git的工作流程（如分支管理）已成为行业标准，掌握Git意味着更好的职业发展机会。无论是个人项目还是团队开发，Git都能显著提升开发效率和质量

4.创建分支及解决问题

先按照要求，创建分支feature，并分别在feature和main分支上对main.c文件进行修改

之后遇到冲突

查询资料后了解到需要手动修改

以下为操作指令及修改结果

```
hswang@localhost:~/lab0-gitlab-hswang24$ git add main.c
git commit -m "在 main 分支上修改 main.c"
[main efcbb7f] 在 main 分支上修改 main.c
1 file changed, 1 insertion(+), 1 deletion(-)
hswang@localhost:~/lab0-gitlab-hswang24$ git merge feature
Auto-merging main.c
CONFLICT (content): Merge conflict in main.c
Automatic merge failed; fix conflicts and then commit the result.
hswang@localhost:~/lab0-gitlab-hswang24$ cat main.c
#include <stdio.h>

int main()
{
    // @TODO: print a sentence you want.
<==== HEAD
    printf("Hello, world!\n");
}Main
=====
    printf("Hello, FDU!\n");
}
Feature
>>>>>> feature
hswang@localhost:~/lab0-gitlab-hswang24$ git status
On branch main
Your branch is ahead of 'origin/main' by 1 commit.
  (use "git push" to publish your local commits)

You have unmerged paths.
  (fix conflicts and run "git commit")
  (use "git merge --abort" to abort the merge)

Unmerged paths:
  (use "git add <file>..." to mark resolution)
    both modified:   main.c

no changes added to commit (use "git add" and/or "git commit -a")
hswang@localhost:~/lab0-gitlab-hswang24$ nano main.c
hswang@localhost:~/lab0-gitlab-hswang24$ git add main.c
hswang@localhost:~/lab0-gitlab-hswang24$ git commit -m"冲突手动修改解决，将两次修改合并"
[main b938c89] 冲突手动修改解决，将两次修改合并
hswang@localhost:~/lab0-gitlab-hswang24$ |
```

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    // @TODO: print a sentence you want.
```

```
    printf("Hello, world!\n");
```

```
}Main
```

```
Feature
```